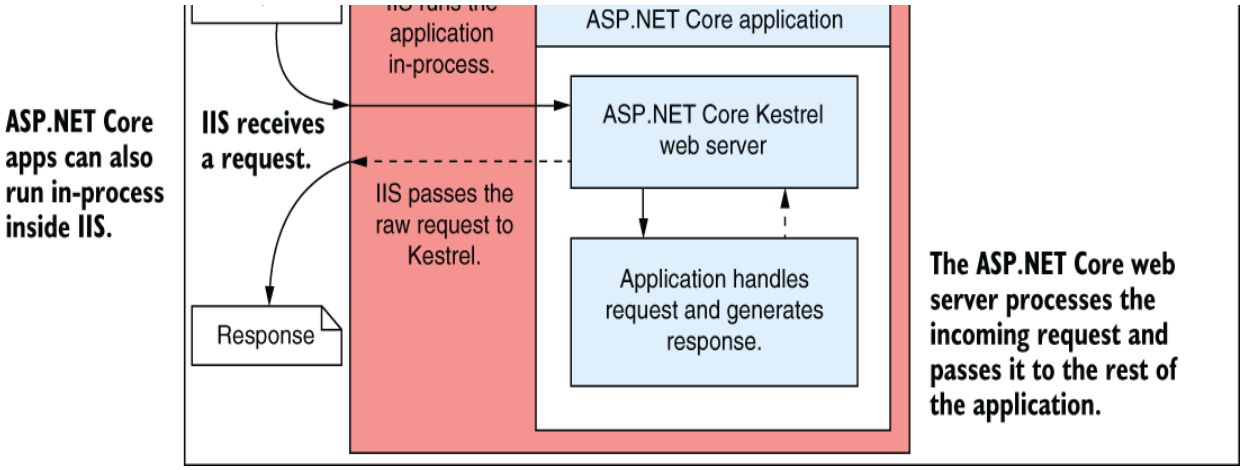


Figure 2.6 The difference between hosting models in ASP.NET (top) and ASP.NET Core (bottom). In the previous version of ASP.NET, IIS is tightly coupled with the application. The hosting model in ASP.NET Core is simpler; IIS hands off the request to a self-hosted web server in the ASP.NET Core application and receives the response but has no deeper knowledge of the application.

NOTE By default, when running in Windows, ASP.NET Core runs *inside* IIS, as shown in figure 2.6, which can provide better performance than the reverse-proxy version. This is primarily a deployment detail and doesn't change the way you build ASP.NET Core applications.

Changing the hosting model to use a built-in HTTP web server has created another opportunity. Performance has been something of a sore point for ASP.NET applications in the past. It's certainly possible to build high-performing applications—Stack Overflow (https://stackoverflow.com) is a testament to that fact—but the web framework itself isn't designed with performance as a priority, so it can end up being an obstacle.

To make the product competitive cross-platform, the ASP.NET team focused on making the Kestrel HTTP server as fast as possible. TechEmpower (https://www.techempower.com/benchmarks) has been running benchmarks